Magíster en Estadística, PUCV

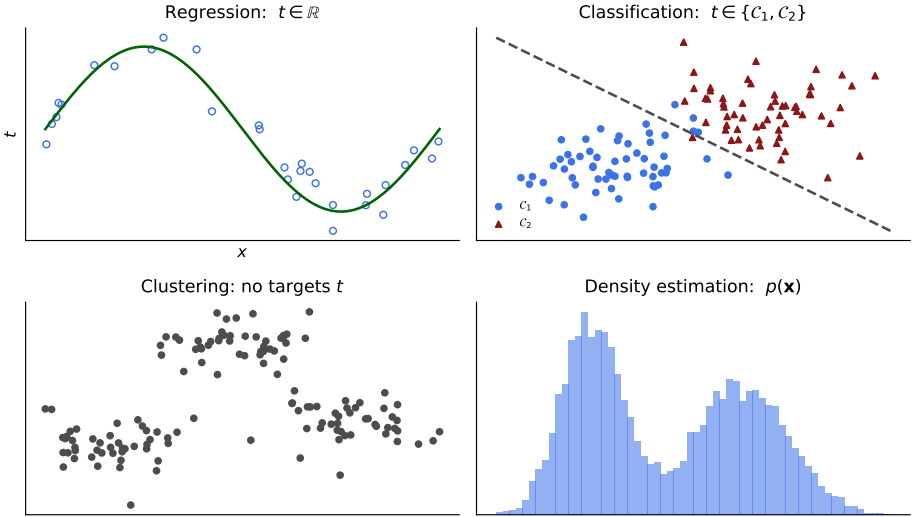
Introduction to Machine Learning

Alejandro Veloz

Overview

1. What machine learning is, and the kinds of problems it solves.
2. Notation: Bishop and Hastie–Tibshirani–Friedman.
3. Supervised learning: loss, risk and the optimal predictor.
4. **The likelihood function**, and where losses come from.
5. The running example: polynomial curve fitting.
6. Overfitting and the bias–variance decomposition.
7. Nearest neighbors and the curse of dimensionality.
8. Model selection: validation and cross-validation.

Motivation


Regression: $t \in \mathbb{R}$
t
x
Classification: $t \in \{C_1, C_2\}$
C_1
C_2
Clustering: no targets t
Density estimation: $p(\mathbf{x})$

Machine learning (ML) is about *making decisions or predictions based on data*.

Machine learning (ML) is about *making decisions or predictions based on data*.

- Arthur Samuel (1959): ML is the field of study that gives computers the ability to learn **without being explicitly programd.**
- Tom Mitchell (1998): A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

The fundamental problem

Given a training set $\mathcal{D} = \{(\mathrm{x}_n, t_n)\}_{n=1}^{N}$, with $\mathrm{x}_n \in \mathcal{X}$ and $t_n \in \mathcal{T}$, find a function $y : \mathcal{X} \to \mathcal{T}$ that

1. approximates the observed data well, and
2. **generalizes** to unseen data.

The fundamental problem

Given a training set $\mathcal{D} = \{(\mathrm{x}_n, t_n)\}_{n=1}^{N}$, with $\mathrm{x}_n \in \mathcal{X}$ and $t_n \in \mathcal{T}$, find a function $y : \mathcal{X} \to \mathcal{T}$ that

1. approximates the observed data well, and
2. **generalizes** to unseen data.

The two goals are in tension. Almost everything in this course is a way of managing that tension.

ML versus neighboring fields

Related fields share techniques but differ in goals:

- In economics and psychology, the goal is to discover the underlying **causal** processes.
- In statistics, the goal is often a model that **fits and explains** a data set, with interpretable parameters.
- In ML, models are often a means to the end of making good **predictions or decisions**.

Notation

We will use two notational systems, because the two reference texts use two.

	Bishop (PRML)	**Hastie, Tibshirani & Friedman** (ESL)
input vector	$\mathbf{x}$, $\mathbf{x}_n$	X, x_i
target	t, t_n	Y, y_i
number of examples	N	N
number of inputs	D	p
parameters	$\mathbf{w} = (w_0, \ldots, w_M)^\mathsf{T}$	$\beta = (\beta_0, \ldots, \beta_p)^\mathsf{T}$
model	$y(\mathbf{x}, \mathbf{w})$	$\hat{f}(X)$
basis / features	$\phi_j(\mathbf{x})$, design $\mathbf{\Phi}$	$h_j(X)$, design $\mathbf{X}$
error / loss	$E(\mathbf{w})$	RSS(β), $L(Y, \hat{f}(X))$
regularization weight	λ	λ (or budget t)

Notation

We will use two notational systems, because the two reference texts use two.

	Bishop (PRML)	**Hastie, Tibshirani & Friedman** (ESL)
input vector	$\mathbf{x}$, $\mathbf{x}_n$	X, x_i
target	t, t_n	Y, y_i
number of examples	N	N
number of inputs	D	p
parameters	$\mathbf{w} = (w_0, \dots, w_M)^\mathsf{T}$	$\beta = (\beta_0, \dots, \beta_p)^\mathsf{T}$
model	$y(\mathbf{x}, \mathbf{w})$	$\hat{f}(X)$
basis / features	$\phi_j(\mathbf{x})$, design $\mathbf{\Phi}$	$h_j(X)$, design $\mathbf{X}$
error / loss	$E(\mathbf{w})$	RSS(β), $L(Y, \hat{f}(X))$
regularization weight	λ	λ (or budget t)

Rule of thumb: **Bishop notation for the probabilistic material, ESL notation for ridge, lasso and model selection.**

Kinds of learning

Supervised. $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}$.

- t continuous $\rightarrow$ regression.
- t categorical $\rightarrow$ classification.

Unsupervised. $\mathcal{D} = \{\mathbf{x}_n\}$.

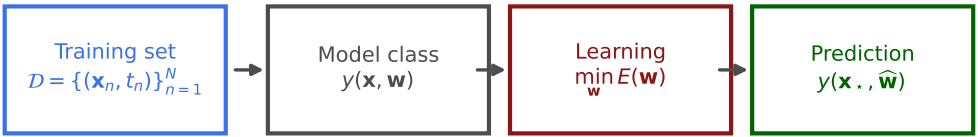
- clustering, density estimation, dimensionality reduction.

Semi-supervised. A few labels, many unlabeled inputs.

Reinforcement. No targets, only a scalar reward from interacting with an environment.

Self-supervised. Targets manufactured from the inputs themselves.

The supervised learning pipeline



generalization is judged on data never seen during learning

Loss and expected loss

Choose a **loss function** $L(t, y(\mathrm{x}))$ measuring the cost of predicting $y(\mathrm{x})$ when the truth is t.

Loss and expected loss

Choose a **loss function** $L(t, y(\mathrm{x}))$ measuring the cost of predicting $y(\mathrm{x})$ when the truth is t.

We would like to minimize the **expected loss** (ESL: the *test* or *generalization* error, Bishop §1.5.5):

$$\mathbb{E}[L] = \iint L(t, y(\mathrm{x}))\, p(\mathrm{x}, t)\, \mathrm{d}\mathrm{x}\, \mathrm{d}t$$

Loss and expected loss

Choose a **loss function** $L(t, y(\mathrm{x}))$ measuring the cost of predicting $y(\mathrm{x})$ when the truth is t.

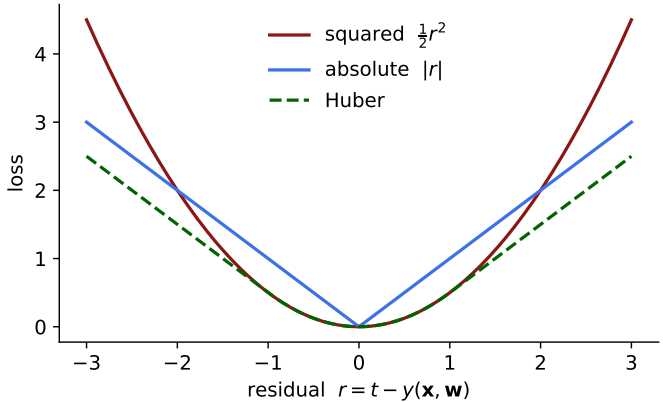
We would like to minimize the **expected loss** (ESL: the *test* or *generalization* error, Bishop §1.5.5):

$$\mathbb{E}[L] = \iint L(t, y(\mathrm{x}))\, p(\mathrm{x}, t)\, \mathrm{d}\mathrm{x}\, \mathrm{d}t$$

What we can actually compute is the **empirical** loss on $\mathcal{D}$:

$$\hat{\mathbb{E}}[L] = \frac{1}{N} \sum_{n=1}^{N} L(t_n, y(\mathrm{x}_n))$$

Common losses



Squared loss is the default for regression: differentiable, and it makes the math tractable. Absolute loss and Huber loss are more robust to outliers.

The optimal predictor under squared loss

Minimizing $\mathbb{E}[L]$ with $L = \{t - y(\mathrm{x})\}^2$ by the calculus of variations gives (Bishop §1.5.5, ESL §2.4):

$$\boxed{y^\star(\mathrm{x}) = \mathbb{E}[t \mid \mathrm{x}] = \int t\, p(t \mid \mathrm{x})\, \mathrm{d}t}$$

The optimal predictor under squared loss

Minimizing $\mathbb{E}[L]$ with $L = \{t - y(\mathrm{x})\}^2$ by the calculus of variations gives (Bishop §1.5.5, ESL §2.4):

$$\boxed{y^\star(\mathrm{x}) = \mathbb{E}[t \mid \mathrm{x}] = \int t\, p(t \mid \mathrm{x})\, \mathrm{d}t}$$

The best possible prediction is the **conditional mean** — the *regression function*.

The optimal predictor under squared loss

Minimizing $\mathbb{E}[L]$ with $L = \{t - y(\mathrm{x})\}^2$ by the calculus of variations gives (Bishop §1.5.5, ESL §2.4):

$$\boxed{y^\star(\mathrm{x}) = \mathbb{E}[t \mid \mathrm{x}] = \int t\, p(t \mid \mathrm{x})\, \mathrm{d}t}$$

The best possible prediction is the **conditional mean** — the *regression function*.

- k-nearest neighbors approximates it by averaging over a neighborhood of x.
- Linear regression approximates it by assuming $\mathbb{E}[t \mid \mathrm{x}]$ is linear in x.

Where does the squared loss come from?

So far we *chose* the squared loss. We can instead **derive** it, by modeling the noise.

Where does the squared loss come from?

So far we *chose* the squared loss. We can instead **derive** it, by modeling the noise.

Assume the target is the model output plus Gaussian noise (Bishop §1.2.5):

$$p(t \mid x, \mathrm{w}, \beta) = \mathcal{N}(t \mid y(x, \mathrm{w}),\ \beta^{-1})$$

Where does the squared loss come from?

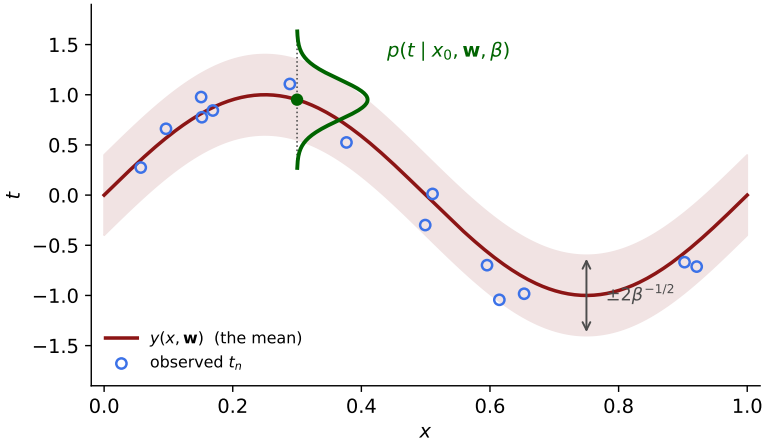
So far we *chose* the squared loss. We can instead **derive** it, by modeling the noise.

Assume the target is the model output plus Gaussian noise (Bishop §1.2.5):

$$p(t \mid x, \mathrm{w}, \beta) = \mathcal{N}(t \mid y(x, \mathrm{w}),\ \beta^{-1})$$

$\beta = 1/\sigma^2$ is the **precision** of the noise. The model no longer predicts a number — it predicts a **distribution**.

The Gaussian noise model



The curve is the **mean** of t at each x; the band is where the model says the data should fall.

The likelihood function

For N observations drawn independently, the probability of the data is the product

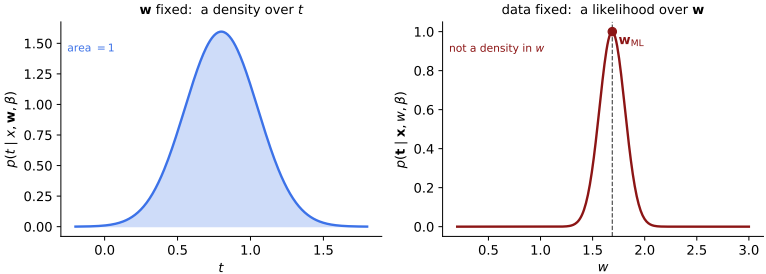
$$p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \beta) = \prod_{n=1}^{N} \mathcal{N}(t_n \mid y(x_n, \mathbf{w}), \beta^{-1})$$

The likelihood function

For N observations drawn independently, the probability of the data is the product

$$p(\mathrm{t} \mid \mathbf{x}, \mathrm{w}, \beta) = \prod_{n=1}^{N} \mathcal{N}(t_n \mid y(x_n, \mathrm{w}), \beta^{-1})$$

Read as a function of w with the data held fixed, this is the **likelihood function**.



Maximum likelihood = least squares

Maximize the log-likelihood — logs turn the product into a sum and never move the maximum:

$$\ln p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi)$$

Maximum likelihood = least squares

Maximize the log-likelihood — logs turn the product into a sum and never move the maximum:

$$\ln p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi)$$

The last two terms do not involve $\mathbf{w}$, and $\beta > 0$ is a positive constant. So

$$\arg\max_{\mathbf{w}} \ln p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \beta) = \arg\min_{\mathbf{w}} \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2$$

Maximum likelihood = least squares

Maximize the log-likelihood — logs turn the product into a sum and never move the maximum:

$$\ln p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi)$$

The last two terms do not involve $\mathbf{w}$, and $\beta > 0$ is a positive constant. So

$$\arg\max_{\mathbf{w}} \ln p(\mathbf{t} \mid \mathbf{x}, \mathbf{w}, \beta) = \arg\min_{\mathbf{w}} \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2$$

Least squares is maximum likelihood under Gaussian noise. And

$$\frac{1}{\beta_{\mathrm{ML}}} = \frac{1}{N} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}_{\mathrm{ML}}) - t_n\}^2.$$

One recipe, many losses

choose $p(t \mid \mathrm{x}, \mathrm{w}) \longrightarrow$ loss $= -\ln p \longrightarrow$ minimize

Distribution of t	**Negative log-likelihood**	**Known as**
Gaussian	$\frac{1}{2}\sum_n (y_n - t_n)^2$	squared loss
Laplace	$\sum_n \lvert y_n - t_n \rvert$	absolute loss
Bernoulli	$-\sum_n [\, t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$	logistic / log loss
Categorical	$-\sum_n \sum_k t_{nk} \ln y_{nk}$	cross-entropy
Poisson	$\sum_n (y_n - t_n \ln y_n)$	Poisson deviance

One recipe, many losses

$$\text{choose } p(t \mid \mathrm{x}, \mathrm{w}) \longrightarrow \text{loss} = -\ln p \longrightarrow \text{minimize}$$

Distribution of t	**Negative log-likelihood**	**Known as**
Gaussian	$\frac{1}{2}\sum_n (y_n - t_n)^2$	squared loss
Laplace	$\sum_n \lvert y_n - t_n \rvert$	absolute loss
Bernoulli	$-\sum_n [\, t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$	logistic / log loss
Categorical	$-\sum_n \sum_k t_{nk} \ln y_{nk}$	cross-entropy
Poisson	$\sum_n (y_n - t_n \ln y_n)$	Poisson deviance

The loss is not a design choice — it is a consequence of what you assume about the data.

Running example: polynomial curve fitting

Bishop §1.1. Generate N points from $\sin(2\pi x)$ plus Gaussian noise, and fit

$$y(x, \mathrm{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

by minimizing the sum-of-squares error

$$E(\mathrm{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathrm{w}) - t_n\}^2$$

Running example: polynomial curve fitting

Bishop §1.1. Generate N points from $\sin(2\pi x)$ plus Gaussian noise, and fit

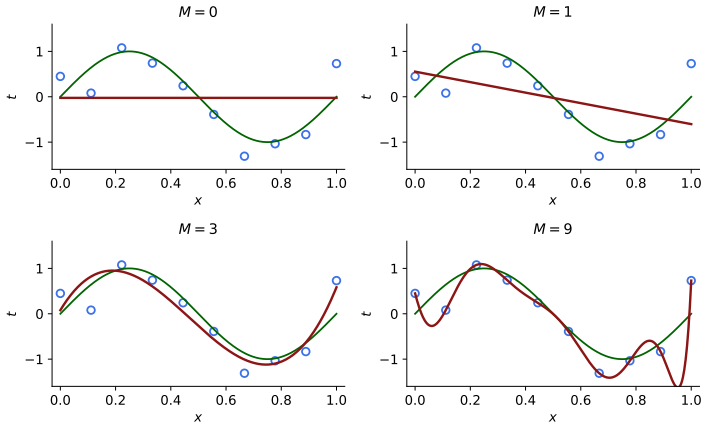
$$y(x, \mathrm{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

by minimizing the sum-of-squares error

$$E(\mathrm{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathrm{w}) - t_n\}^2$$

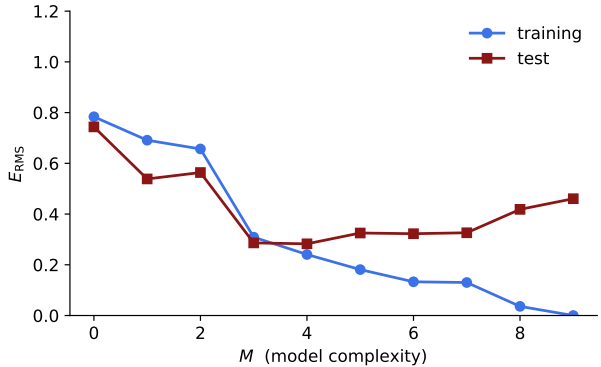
The model is **nonlinear in** x but **linear in** w — so least squares has a closed-form solution.

Choosing M: model selection in miniature



Green: the true $\sin(2\pi x)$. Red: the fitted polynomial. Circles: the $N = 10$ training points.

Training error versus test error



$$E_{\text{RMS}} = \sqrt{2E(\mathrm{w}^\star)/N}$$

What overfitting does to the coefficients

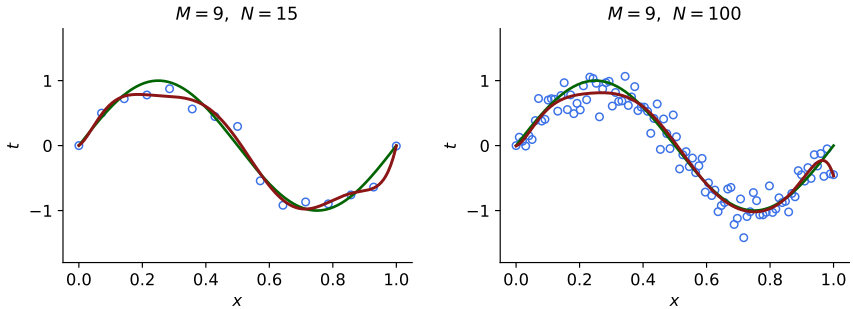
	$M=0$	$M=1$	$M=3$	$M=9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$\vdots$				$\vdots$
$w_9^\star$				125201.43

What overfitting does to the coefficients

	$M=0$	$M=1$	$M=3$	$M=9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$\vdots$				$\vdots$
$w_9^\star$				125201.43

Overfitting shows up as coefficients of enormous magnitude and alternating sign.

More data cures overfitting

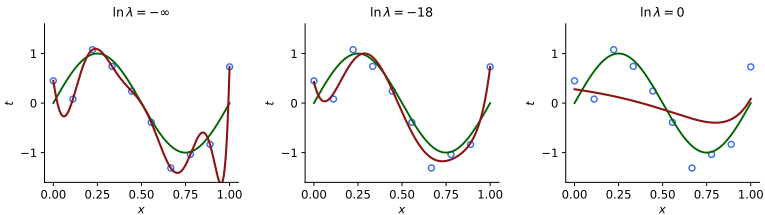


The same $M = 9$ model. A rough (and much-abused) heuristic: keep N several times the number of parameters.

Regularization: a first look

Penalize large coefficients by adding a term to the error (Bishop §1.1, ESL §3.4):

$$\widetilde{E}(\mathrm{w}) = \underbrace{\frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathrm{w}) - t_n\}^2}_{\text{fit}} + \underbrace{\frac{\lambda}{2}\|\mathrm{w}\|_2^2}_{\text{complexity}}$$



The bias–variance decomposition

For squared loss, with $t = h(\mathbf{x}) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\text{var}(\varepsilon) = \sigma^2$, and $\mathcal{D}$ random (ESL §7.3, Bishop §3.2):

$$\mathbb{E}_{\mathcal{D}}[(t - y(\mathbf{x}; \mathcal{D}))^2] = \underbrace{(\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})] - h(\mathbf{x}))^2}_{(\text{bias})^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(y(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}; \mathcal{D})])^2]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

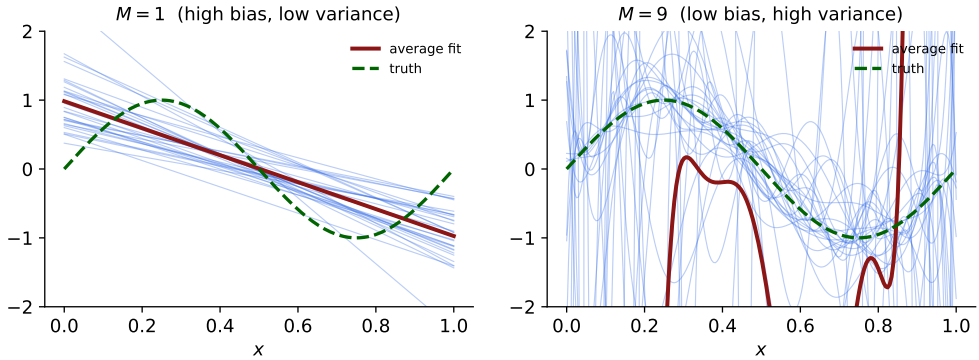
The bias–variance decomposition

For squared loss, with $t = h(\mathbf{x}) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\mathrm{var}(\varepsilon) = \sigma^2$, and $\mathcal{D}$ random (ESL §7.3, Bishop §3.2):

$$\mathbb{E}_{\mathcal{D}}\big[(t - y(\mathbf{x};\mathcal{D}))^2\big] = \underbrace{\big(\mathbb{E}_{\mathcal{D}}[y(\mathbf{x};\mathcal{D})] - h(\mathbf{x})\big)^2}_{(\text{bias})^2} + \underbrace{\mathbb{E}_{\mathcal{D}}\big[(y(\mathbf{x};\mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x};\mathcal{D})])^2\big]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

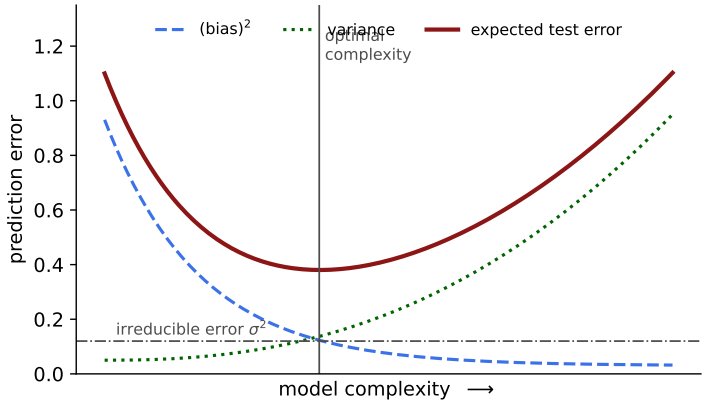
- **Bias**: error from the model class being too rigid to represent h.
- **Variance**: error from sensitivity to the particular training set drawn.
- **Noise**: irreducible; no model can beat it.

Bias and variance, seen directly



Thirty fits, each from an independent training set of $N = 12$ points.

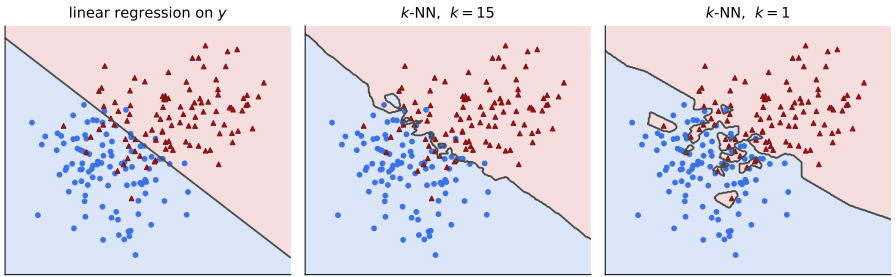
The trade-off



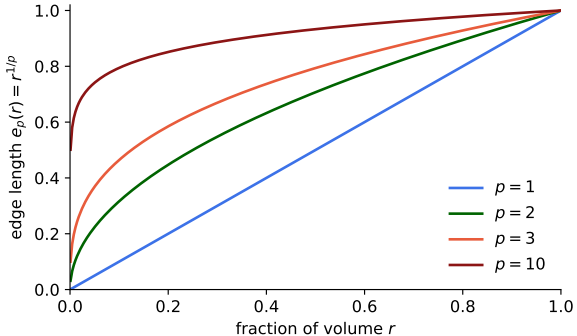
Nearest neighbors

$$\hat{f}(x) = \frac{1}{k} \sum_{x_i \in N_k(x)} y_i$$

where $N_k(x)$ is the set of the k training points closest to x (ESL §2.3.2).

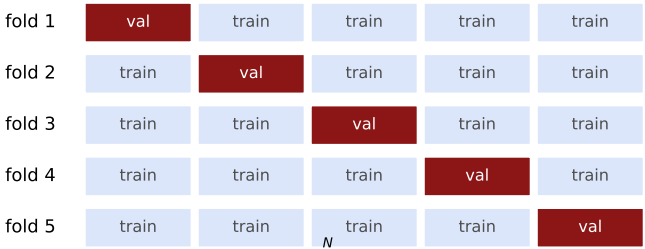


The curse of dimensionality



To capture a fraction r of the unit hypercube in p dimensions, a sub-cube needs edge length $e_p(r) = r^{1/p}$. In $p = 10$, capturing 1% of the volume needs 63% of the range of *each* coordinate.

Model selection: hold-out and cross-validation



K-fold CV (ESL §7.10): split $\mathcal{D}$ into K folds, train on $K-1$, validate on the one left out, average. Typical K: 5 or 10.

Practical rules

1. **Split before you look.** Hold out a test set and touch it once.
2. **Standardize inputs** when the method is not scale invariant (ridge, lasso, k-NN, anything with a penalty).
3. **Preprocess inside the fold**, never before splitting.
4. **Match the loss to the problem**, not to convenience.
5. **Compare against a baseline** — the mean of t, or a single-feature model. Many published gains do not survive this.
6. **Report variability**, not a single number: CV standard errors, repeated splits.

Summary

- The optimal predictor under squared loss is the conditional mean $\mathbb{E}[t \mid \mathrm{x}]$.
- **Losses come from likelihoods**: $-\ln p(t \mid \mathrm{x}, \mathrm{w})$. Gaussian $\Rightarrow$ least squares.
- Training error is not generalization error; the gap grows with model complexity.
- Test error decomposes into $(\text{bias})^2 + \text{variance} + \sigma^2$.
- Overfitting manifests as large, cancelling coefficients $\Rightarrow$ **penalize them**.
- Choose complexity by cross-validation, not by training error.

Summary

- The optimal predictor under squared loss is the conditional mean $\mathbb{E}[t \mid \mathrm{x}]$.
- **Losses come from likelihoods**: $-\ln p(t \mid \mathrm{x}, \mathrm{w})$. Gaussian $\Rightarrow$ least squares.
- Training error is not generalization error; the gap grows with model complexity.
- Test error decomposes into $(\text{bias})^2 + \text{variance} + \sigma^2$.
- Overfitting manifests as large, cancelling coefficients $\Rightarrow$ **penalize them**.
- Choose complexity by cross-validation, not by training error.

Next: ridge and lasso — two ways of penalizing w, with very different consequences.